

Student ID: _____
Student Name: _____

Dhirubhai Ambani Institute of Information & Communication Technology
Second In-Semester Examination, Autumn Semester 2023-2024

Course Title IT227 Computer Systems Programming
Date 12th Oct 2023

Max Marks: 70
Time: 2 Hours

Instructions:

1. All questions are compulsory.
2. Write the answers for all questions in the space provided in -the question paper only.
3. Write your answers in brief. Writing long answers does not fetch higher marks.
4. Use of calculators is allowed.

Q1. Short Answer 1-3 lines [3 Marks each]

a. In the code provided below assume that `thr_fn` exists and is defined as `void *thr_fn(void *arg)`. How many processor cores will be required to run the code if threads were implemented as user-level threads (ULT)? If the implementation is changed to kernel-level threads (KLT), will the required number of processor cores change? why?

```
main() {  
    fork();  
    pthread_create(&threaded, NULL, thr_fn, NULL);  
}
```

Answer:

For ULT 2 cores are required because only two processes will be allocated a separate cores.

For KLT 4 cores will be required because there is a default thread for each of the two process i.e. 2 threads and two more threads will be created by executing `pthread_create` by both processes totaling 4 threads.

b. Consider that an application reads an array of 100 integers each with 4 bytes and performs at least 8 operations on each integer. Which type of application is this? How threading can be used to improve the performance of this type of application?

Answer:

Compute-bound. By allowing to perform operations in threads.

c. What will be the output from the code below? Explain you answer briefly.

```
void *thr_fn(void *arg) {  
    int *ret = (int *)malloc(sizeof(int));  
    *ret = getpid();  
    return ((void *) ret);  
}  
  
int main() {  
    pthread_t tid;  
    int *tret;  
    pthread_create(&tid, NULL, thr_fn, NULL);  
    pthread_join(tid, (void **)&tret);
```

```

    printf("%d\n", (int)*tret + getpid());
    free(tret);
    pthread_exit(0);
}

```

Answer:

It will display the 2*processed because both threads are part of the same process hence the process id in main thread and child thread are same. In child thread 'return' will allow the main thread to continue to run even after termination of child thread.

- d. Please indicate if there will be a race condition in the code and for which variable? Explain briefly.

```

int i *myid;
void *foo(void *vargp) {
    myid = ((int *)vargp);
    *myid = *myid + 10;
}
int main() {
    pthread_t tid[2];
    for (i = 0; i < 2; i++)
        pthread_create(&tid[i], NULL, foo, &i);
    for (i = 0; i < 2; i++)
        pthread_join(tid[i], NULL);
}

```

Answer:

myid variable will cause the race condition since address of i variable is passed to thread function and it is possible that both thread will execute the statement *myid = *myid + 10 at the same time.

- e. Consider code below. What is the expected outcome when it is executed? Explain briefly.

<pre> int x, i; sem_t sem; void *fn1(void *arg) { sem_wait(&sem); fn2(); x = x+1; sem_post(&sem); } </pre>	<pre> void fn2() { sem_wait(&sem); x = x+1; sem_post(&sem); } </pre>	<pre> int main() { pthread_t tid[2]; sem_init(&sem, 0, 1); for (i = 0; i < 2; i++) pthread_create(&tid[i], NULL, fn1, NULL); for (i = 0; i < 2; i++) pthread_join(tid[i], NULL); sem_destroy(&sem, 0, 1); } </pre>
--	--	--

Answer:

Deadlock situation since any one of the threads get a lock on semaphore in fn1 and then it tries to get lock on the same semaphore.

- f. For each of these codes, what are all possible counter variables values that can be printed? Briefly explain your answer.

<pre>// code1 int counter = 5; void foo() { pthread_mutex_lock(&m); counter=counter+2; printf("%d", counter); pthread_mutex_unlock(&m); } int main() { pthread_t tid[2]; int i; pthread_mutex_init(&m, NULL); for (i = 0; i < 3; i++) { pthread_create(&tid[i], NULL, foo, NULL); } for (i = 0; i < 3; i++) { pthread_join(tid[i], NULL); } pthread_mutex_destroy(&m); counter=counter+2; printf("%d", counter); } }</pre>	<pre>// code2 int counter = 5; void foo() { counter=counter+2; printf("%d", counter); } int main() { int i; for (i = 0; i < 2; i++) { fork(); foo(); } counter=counter+2; printf("%d", counter); } }</pre>
--	---

Answer:

Code1 will print 7, 9, 11, 13 in any order since three threads will increment the value from 5 to 7 to 9 to 11 and main thread finally increment value to 13.

Code2 will print 7, 9 in parent and child processes.

g. For each situation, state the one primitive that, when used correctly around the relevant critical section, prevents race conditions and results in the most concurrency. When more than one primitive will work with equal concurrency, give the primitive that is simplest, as defined below. If your answer is a semaphore, you must specify its initial value. Your response to each question will be exactly one of the following primitives, listed here in order from simplest to most complex: none, mutex, semaphore(n), rwlock.

Situation:	Answer:
A. Two threads update a global variable.	mutex
B. Five threads search through a regular linked list of integers.	none
C. Two hundred threads search through a regular linked list of integers; one thread occasionally removes and frees nodes from the list.	rwlock
D. At most, seven threads may simultaneously be within the critical section using 7 shared resources.	sem(7)

E. The operating system maintains the process table, which can be read by several threads, but only the main kernel thread is allowed to create a new process or remove the finished process in the process table.	rwlock
F. Google Chrome browser has two tabs created as threads trying to use 2 ethernet ports to communicate.	sem(2)

h. Consider thr_fn1 and thr_fn2 functions run in two separate threads. Assume that semaphore (sem) is initialized with value 0. How many possible values of x will be printed after both threads complete execution?

```
int x=5;
void *thr_fn1(void *arg) {
    int y;
    y = x*2;
    x = y;
    sem_post(sem)
}
void *thr_fn2(void *arg) {
    int z;
    sem_wait(sem);
    z = x+1;
    x = z;
    printf("%d", x)
}
```

Answer:

Only one since thread 2 cannot execute before thread1 finishes. Thread1 will change the value of x = x*2 i.e. 5*2=10 and then thread2 will increment x i.e. x=x+1. hence the value of x is only 11.

Q2:

Consider the producer-consumer problem where a circular queue is implemented using an array and filled by the producer as the items are produced and the consumer consumes items from the queue. Producers will have to wait when the queue is full i.e., no new items can be placed in the queue. Consumers will have to wait if the queue is empty until the producer adds a new item to the queue. The queue accommodates at most 100 items. Partial code is provided below; fill in the blanks by using counting semaphores within producer and consumer functions so that it waits for the respective conditions. Also, perform semaphore initialization in the main function. NOTE: The assumption is that there are two producer threads and two consumer threads accessing the queue and produce_item, place_item_in_queue, consume_item and remove_item_from_queue are all implemented. [6 Marks]

```
int queue[100];
void *producer(void *arg) {
    sem_wait(slots);
```

```
    item = produce_item();  
    place_item_in_queue(item);
```

```
sem_wait(&items);  
}
```

```
void *consumer(void *arg) {  
sem_wait(&items);
```

```
    item = remove_item_from_queue();  
    consume_item(item);
```

```
sem_wait(&slots);
```

```
}  
main() {
```

```
sem_init(&items, 0, 0);  
sem_init(&slots, 0, 100);
```

```
    int tid1, tid2, tid3, tid4;  
    pthread_create(&tid1, NULL, producer, NULL);  
    pthread_create(&tid2, NULL, producer, NULL);  
    pthread_create(&tid3, NULL, consumer, NULL);  
    pthread_create(&tid4, NULL, consumer, NULL);
```

```
sem_destroy(&items);
sem_destroy(&slots);

}
```

Q3: Assume segmentation is used as a virtual memory mechanism. In a 16-bit address space, code, heap and stack begin at 1KB, 8KB and 64KB, respectively. The following table shows the segment size and the corresponding base register. [4 Marks]

Segment	base	Size
Code	21 KB	2 KB
heap	40 KB	2 KB
stack	28 KB	2 KB

- a. What is the correct address translation for following virtual addresses? [1.5 Marks]
- i. 9272

Answer 42040

- ii. 64000

Answer 27136

- b. Compared to dynamic relocation using a single base and bounds register, what is the proportion of physical memory saved while using segmentation for address translation. [1.5 marks]

Answer: 58/64

- c. What is the valid range of virtual addresses that may be returned on a call to malloc()? [1 marks]

Answer: 8KB to 10KB-1 or 8192 - 10239)

Q4: Answer briefly. [3 Marks]

- a. Consider the following code snippet. What is the total size of memory allocated? (1.5 marks)

```
void main()
{
    int x = 1024;
    int *ptr;
    while(int i=x; i>1; i>>1)
    {
        ptr = (int *) malloc(10*sizeof(int));
    }
}
```

Answer: 10*40

- b. What is/are the difference(s) between these two system calls [1.5 marks]

- i. `mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_ANON|MAP_PRIVATE, -1, 0);`
- li. `mmap(0x4000, 4096, PROT_READ|PROT_WRITE, MAP_ANON|MAP_PRIVATE, -1, 0);`

Answer: first call leaves up to the OS to decide where the memory should be allocated. Second call requests memory allocation near the address specified as first argument.

Q5: Write code to print the heap break and increment the heap memory by 10KB. [3 marks]

#include

Answer

```
#include <stdlib.h>
#include <stdio.h>
Void main()
{
printf("heap break at : %p\n", (void *) sbrk(0));
void *ptr = (void *)sbrk(10240);
}
```

Q6: Consider the following pseudocode for the paging mechanism. Assume a 24 bit address space with 512 byte sized pages. Assume 4 bytes per entry of page table and 20-bit page frame number.

```
VPN = (VirtualAddress & A) >> B
PTEAddr = PTBR + (VPN * sizeof(PTE))
PTE = AccessMemory(PTEAddr)
if (PTE.Valid == False)
    RaiseException(SEGMENTATION_FAULT)
else if (CanAccess(PTE.ProtectBits) == False)
    RaiseException(PROTECTION_FAULT)
else
    // Access is OK: form physical address and fetch it
    offset = VirtualAddress & C
    PhysAddr = (PTE.PFN << D) | offset
    Register = AccessMemory(PhysAddr)
```

What is the value of the following

[7 marks]

- a. number of pages (**Answer: $2^{15} = 32\text{KB} = 32768$**)

b. size of the page table (**Answer: $2^{17} = 131072$ bytes**)

c. A in hexadecimal **0xFFFE0**

d. B in hexadecimal **0x11 or 9**

e. C in hexadecimal **0x0001F**

f. D in hexadecimal **same as B**

g. What is the size of the supported physical memory?

Q7: Consider a hypothetical 2KB free heap memory starting at address 1024 as shown below. Assume that memory allocation and deallocation is done from the memory. Consider the following code snippet and answer the following questions. [8 Marks]

```
node_t head = mmap(NULL, 2048, PROT_READ|PROT_WRITE, MAP_ANON|MAP_PRIVATE, -1, 0);
int *ptr1 = (int *) malloc(256);
int *ptr2 = (int *) malloc(64);
free(ptr2);
```

a. How many allocated and free blocks of memory are there? [1 marks]

Answer(Allocated: 1) and free(2)

b. What is the size of the free memory? [1 marks]

Answer: 1704

c. What are the values of ptr1, ptr2, head, head->size, head -> next [4 marks]

Answer: ptr1= 1024+8=1032,

ptr2 =ptr1+256+8=1296,

head=1288,

head->size: 64, head->next=1360,

d. After call to free(ptr1) What is the value of head, head->size and head->next? [2 marks]

Answer: head= 1024, head->size = 256, head-next = 1288

Q8: Consider the page table below. If all the pages of the process with ASID 2 are accessed one by one. [6 Marks]

VPN	PFN	Valid bit	ASID	Present bit	Use bit	Dirty bit
100	901	0	1	0	0	1
101	902	1	2	0	1	1
100	909	0	2	1	0	0
102	909	0	1	1	1	1
104	2000	1	2	0	0	1

- a. Accessing which page frame(s) will lead to segmentation fault? [1.5 marks]

Answer: PFN- 909

- b. How many page faults can be inferred from the table? [1.5 marks]

Answer 2

- c. Which page will be replaced by the clock algorithm? [1.5 marks]

VPN-100 PFN-909

- d. Are there any shared pages? If yes, what are their VPNs? [1.5 marks]

Answer: 100 and 102

Q9: Consider the following assembly instructions, how many memory references are generated for each of the instructions? Assume paging is used for address translation. Page table mappings are shown in the table below. Page table is at PFN 10. Assume 8 bit address space starting at 0, page size of 64 bytes and 4 bytes page table entry. Answer the following: [6 marks]

124 : movl 0x0(200), %eax ; load content at (0+200) into eax

128 : addl \$0x03, %eax; add 3 to eax register

132 : movl %eax, 0x0(%ebx) ; store eax back to mem

VPN	PFN
0	2
1	3
2	4
3	5

a. What is (are) the VPN of the assembly instructions? [2 marks]

Answer VPN 1 (Instruction 1) and VPN 2 (instructions 2 and 3)

b. How many memory references are generated for the first instruction? Justify. [2 marks]

Answer: 4 memory references, PT[2] Mem[PA1], PT[3], Mem[PA2]

c. What is the physical address of the third instruction in the page table? [2 marks]

Answer: $64 \times 4 + 4 = 260$

Q10: Consider the reference stream 0 1 2 0 1 3 4 3 5 2 1 0, LRU replacement policy and cache size of 4. What is the number of page faults and the average memory access time given memory access time of 10 ns and disk access time of 10 ms. [3 Marks]

Answer : 9 page faults. AMAT = 7.5000025 ms